



**Institut Supérieur d'Informatique
et de Mathématiques de Monastir**



SUPPORT DE COURS

Fondements IA

Année Universitaire 2025-2026

Enseignant
Nafaa Haffar

Chapitre 3 : Résolution des problèmes et algorithmes de recherche

- ✓ Définition formelle d'un problème
- ✓ Recherche non-informée
- ✓ Recherche Heuristique
- ✓ Recherche locale

Agents guidés par un but

- ▶ Depuis toujours, on cherche à résoudre des problèmes simples dans leur définition mais difficiles à résoudre.
- ▶ Trouver la bonne succession d'étapes pour atteindre un but donné
- ▶ Exemples :
 - Planifier un trajet
 - Résoudre un
 - casse-tête Jouer
 - etc.
- ▶ Une IA reste un programme informatique, donc est basée sur un algorithme, de calculs à partir de données.
- ▶ Utiliser des algorithmes d'exploration.



D'emarche g'en'erale

- ▶ Passage d'un 'enonc'e informel `a une sp'ecification pr'ecise.
- ▶ Trouver une bonne repr'esentation du probl`eme.
- ▶ Trouver des op'érateurs pour manipuler cette repr'esentation.
- ▶ Effectuer un contr`ole de strat'egie.
- ▶ Recherche d'une solution `a l'int'erieur. du cadre d'efini par ces sp'ecifications.



Définition formelle d'un problème

- Le but de ce chapitre est d'introduire des algorithmes de recherche pour la résolution des problèmes.
- Un problème sera défini par :
 1. Un **état** initial
 2. Un ensemble **d'actions**
 3. Une fonction de **successeur**, qui définit l'état résultant de l'exécution
 4. D'une action dans un état
 5. Un ensemble **d'états buts**
 6. Une **fonction de coût**, associant à chaque action un nombre non-négative (le coût de l'action)

Formuler correctement le problème revient à définir :

- ▶ l'**état** : description du monde relativement au problème considéré ;
- ▶ l'**espace des états** (ou espace de recherche) : ensemble de tous les états possibles ;
- ▶ l'**action** : opération permettant de changer d'état ;
- ▶ un **modèle de transition** ;
 - une fonction qui renvoie l'état résultant de l'exécution d'une action dans un état ;
 - successeur : désigne tout état accessible à partir d'un état donné avec une seule action.
- ▶ La **stratégie** : choix de la prochaine action à entreprendre dans le processus d'exploration
- ▶ Un **test de but** détermine si un état est un état but

Objectifs

- Comment ?
 - Modéliser un Problème
 - Résoudre des problèmes
- Comment ?
 - Passer d'un état à un autre par l'application d'une action donnée.
 - Développer un arbre de recherche.
 - Appliquer une stratégie de recherche.
- Comment ?
 - Rechercher des solutions.
 - Recenser les états d'un système donné et trouver parmi ces états une ou plusieurs solutions.

Objectifs

- **On a un problème Quand ?**

Objectifs

- **On a un problème Quand ?**

- On se trouve dans une situation : **état**
 - On a un but à atteindre : **état avec des caractéristiques données**
 - On ne voit pas la suite d'actions à accomplir pour atteindre ce but : résolution de problèmes
- **La résolution s'effectue en progression d'une hypothèse initiale jusqu'à le but.**

Exemple de Recherche

- Les états sont représentés par des pierres.
- Formuler le problème à résoudre :
- Trouver une stratégie permettant de partir de l'état initial P_1 et de parcourir l'espace des états pour atteindre l'état final P_f .
- Le mouvement entre les pierres (états) est effectué en utilisant une action appelée "sauter".
- Certains états sont accessibles, tandis que d'autres ne le sont pas.
- A chaque état P_i on associe P_j qu'on peut atteindre à partir de P_i .
- On peut se tromper et arriver sur une pierre à partir de laquelle nos forces nous interdisent d'atteindre la suivante.
- On peut revenir en arrière et essayer un autre chemin.
- Chaque action (sauter) est censée rapprocher de l'état final P_f .



Machine déterministe

- L'Humain résout un problème, la machine exécute la solution.
- Pour une entrée donnée et un état initial fixe, produit toujours le même résultat.
- Machine/Système non intelligent.
 - Son comportement est entièrement déterminé par ses règles de transition.
 - Machine déterministe à états finis.

Machine non déterministe

- L'Humain (définir le problème), la machine (Résoudre).
- Système de résolution (automatique) de Problèmes.
 - Pour un même état et une même entrée, peut suivre plusieurs chemins possibles d'exécution.
 - Son comportement n'est pas totalement déterminé à l'avance.
 - Espace de recherche (espace d'états, de buts..).
 - Stratégies de recherche; exploration de l'espace.
 - Prise de décision, devant différents choix.

Exemples de problèmes

- **Le taquin (1: 8-Puzzle)**

- Est une sorte de puzzle.
- Une grille 3×3 de neuf cases où sont placées huit tuiles étiquetées par les nombres 1 à 8, une des cases restant vide.
- Réorganiser les tuiles numérotées dans un ordre précis en effectuant des déplacements successifs en utilisant une case vide.
- On peut déplacer une tuile vers la case vide uniquement (haut, bas, gauche ou droite).

7	2	4
5		6
8	3	1

État initial

Exemples de problèmes

Formalisation Possible de ce problème : Le taquin (1: 8-Puzzle). Un problème sera défini par :

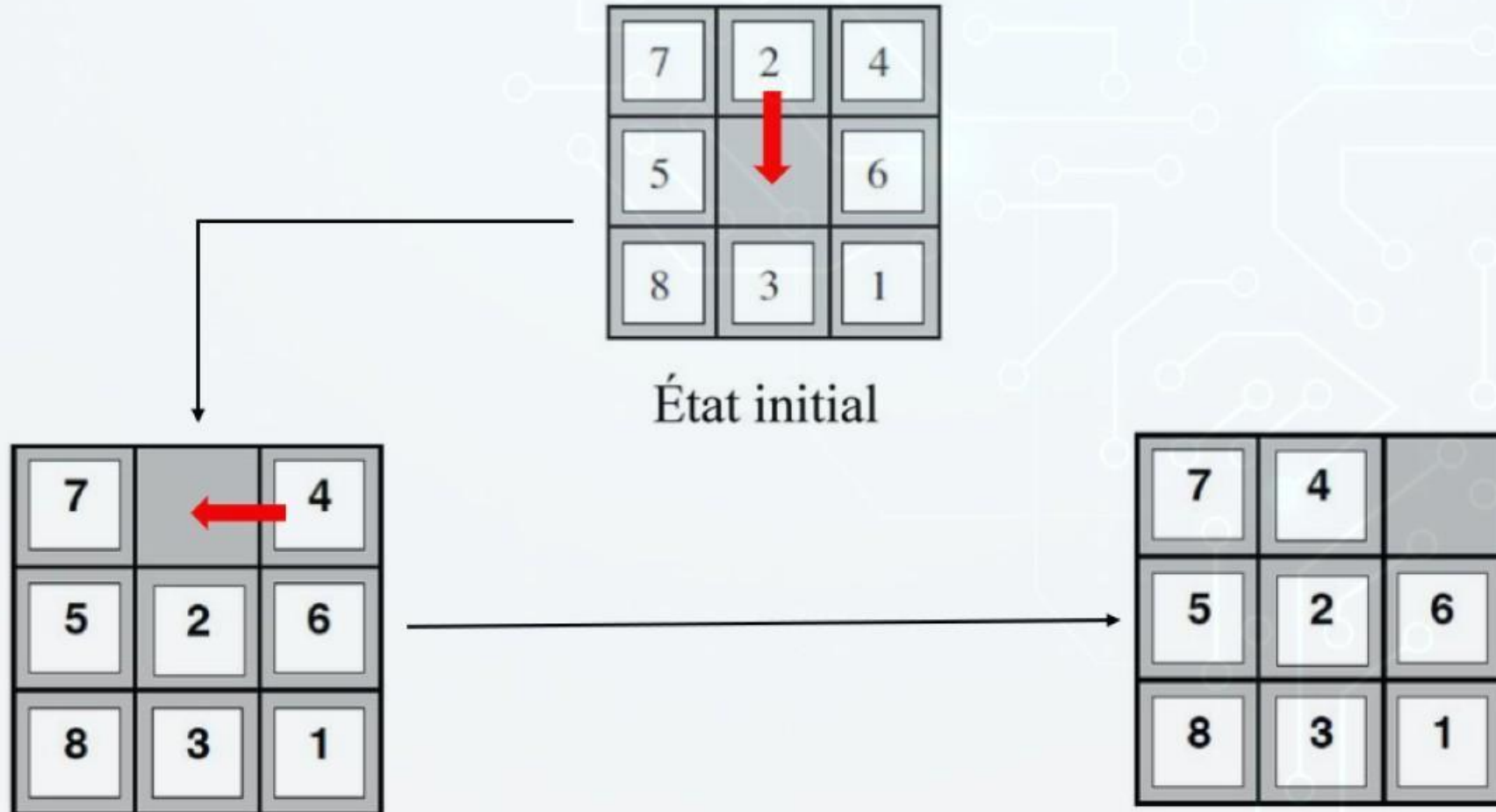
- **Etats** : les configurations des huit tuiles dans les neuf cases de la grille.
 - **Etat initial** : N'importe quel état.
 - **Actions** : Il y aura 4 actions possibles correspondant aux quatre façons de changer la position du carre vide : haut, bas, gauche, droite (remarquez que dans certaines configurations, il n'y aura que 2 ou 3 actions possibles).
 - **Fonction de successeur** : spécifie les états résultants des différentes actions. Par exemple, la fonction va nous dire que l'exécution de l'action droite dans le premier état produira le deuxième état.
 - **Test de but** : L'état but est unique et fixe au début du jeu (en pratique il s'agit de remettre les nombres dans l'ordre mais tout état peut être choisi comme état but, tant qu'il est atteignable).
 - **Coût des actions** : Chaque déplacement d'une tuile a coût de 1 (pour trouver une solution avec le moins de déplacements).
1. Un **état** initial
 2. Un ensemble **d'actions**
 3. Une fonction de **successeur**
 4. D'une action dans un état
 5. Un ensemble **d'états buts**
 6. Une **fonction de coût**

7	2	4
5		6
8	3	1

État initial

Exemples de problèmes

Formalisation Possible de ce problème : Le taquin (1: 8-Puzzle)



Exemples de problèmes

Toutes les configurations du taquin ne sont **pas toujours atteignables**. *

Un principe fondamental du jeu de taquin est que **seule la moitié des configurations possibles sont résolubles**.

Pourquoi ?

- Chaque mouvement du jeu de taquin correspond à une permutation des tuiles en échangeant la case vide avec une tuile adjacente.
- Un invariant mathématique du taquin dit qu'on ne peut atteindre **que les configurations ayant la même parité d'inversions** que l'état initial.
- Cela signifie que **seulement la moitié des 9! configurations sont accessibles à partir d'un état initial donné**.

$$\frac{9!}{2} = \frac{362880}{2} = 181440$$

Les algorithmes d'aujourd'hui arrivent à résoudre les taquins 3×3 et 4×4 (qui ont des espaces d'états de taille 181440 et d'environ 1,3 milliard respectivement), mais les instances du taquin 5×5 (avec un espace d'états de taille 1025) restent difficiles

Exemples de problèmes

Formalisation Possible de ce problème : Le taquin (1: 8-Puzzle)

7	2	4
5		6
8	3	1

État initial

7		4
5	2	6
8	3	1

7	4	
5	2	6
8	3	1

...

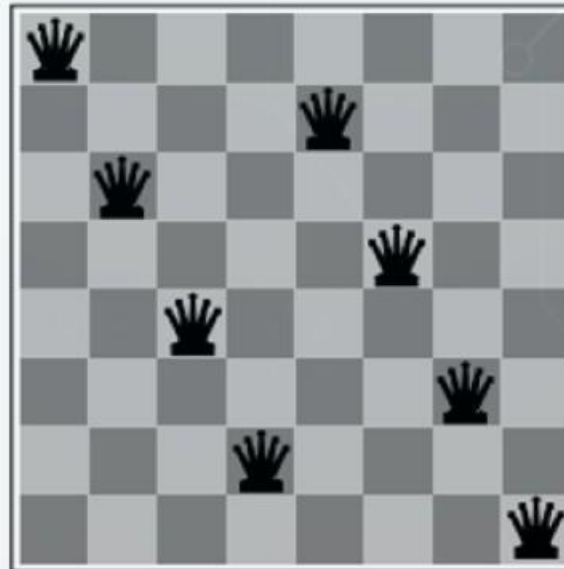
	1	2
3	4	5
6	7	8

État final

Exemples de problèmes

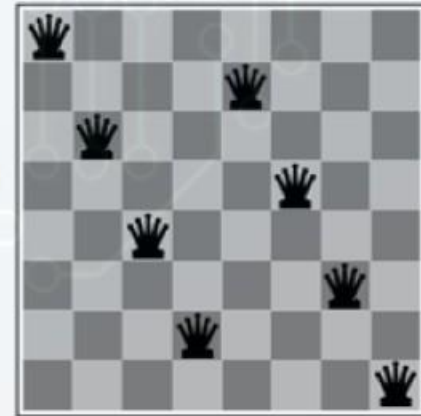
- **Huit reines (8-Reines)**

- Placer huit reines sur un échiquier (une grille 8×8) tel qu' aucune reine attaque une autre reine
- Il n'y a pas deux reines sur la même colonne, la même ligne, ou sur la même diagonale.



Exemples de problèmes

- **États** : Chaque configuration de reines placées sur l'échiquier est un état.
- **État initial** : échiquier vide.
- **Actions** : Ajouter une reine dans une case vide sur l'échiquier. Cette action consiste à trouver une case vide où une reine peut être placée sans qu'elle entre en conflit avec les reines déjà placées.
- **Fonction de successeur** : Cela décrit la configuration de l'échiquier qui résulte de l'ajout d'une nouvelle reine dans la configuration courante.
- **Test de but** : il n'y a pas de conflits entre les reines.
- **Coûts des actions** : Le coût d'ajouter une reine pourrait être soit nul, soit constant mais cela n'est pas nécessaire ici, car on se concentre uniquement sur l'atteinte de l'état final.

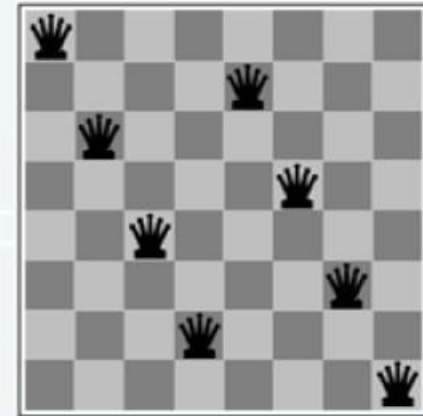


Exemples de problèmes

Formalisations Possibles de ce problème : Huit reines (8-Reines)

1- Méthode incrémentale (Placer les reines une par une)

- **État initial** : Un échiquier vide.
- **Action** : Ajouter une reine à une case vide dans la colonne la plus à gauche, en évitant tout conflit avec les autres reines.
- **Avantage** : À chaque ajout, on vérifie immédiatement si la configuration est valide (sans conflits). Dès que toutes les reines sont placées sans conflit, on atteint l'état final.

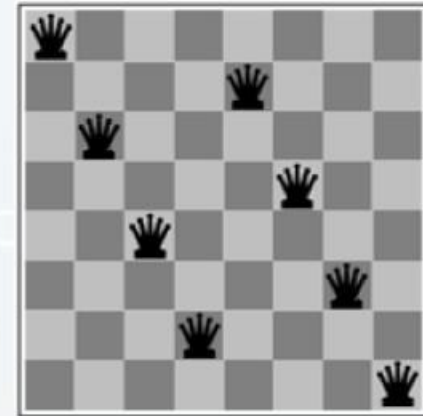


Exemples de problèmes

Formalisations Possibles de ce problème : Huit reines (8-Reines)

2- Méthode aléatoire avec réarrangement (Placer toutes les reines d'abord, puis changer leurs positions)

- **État initial** : On commence avec un échiquier où une reine est placée dans chaque colonne, mais les positions ne sont pas nécessairement correctes.
- **Action** : Changer la position d'une reine dans sa colonne pour éviter les conflits.
- **Problème potentiel** : Il y a un risque de **bouclage** (c'est-à-dire revenir à une configuration déjà visitée). Pour éviter cela, il faut vérifier que l'on ne revisite pas les mêmes configurations.



Exemples de problèmes

Comparaison

Le problème des huit reines est un excellent exemple de **problème de satisfaction de contraintes** où l'on cherche à satisfaire une série de conditions (pas de conflit entre les reines).

Contrairement au **problème du taquin**, où l'on cherche à atteindre un état spécifique en suivant un chemin, le problème des huit reines se concentre uniquement sur la configuration des reines.

Exemples de problèmes

- **Robot aspirateur**

- Problème classique en intelligence artificielle.
- Un agent (le robot) se déplace pour nettoyer efficacement un environnement tout en évitant les obstacles.
- Le problème peut être vu sous plusieurs angles : ??

Exemples de problèmes

- **Robot aspirateur**

Le problème peut être vu sous plusieurs angles :

- **Exploration** : Comment couvrir toute la surface ?
- **Planification** : Quel est le meilleur chemin pour minimiser le temps de nettoyage ?
- **Évitement d'obstacles** : Comment réagir lorsqu'il rencontre un mur ou un meuble ?
- **Autonomie** : Quand et comment retourner à sa station de charge ?

Exemples de problèmes

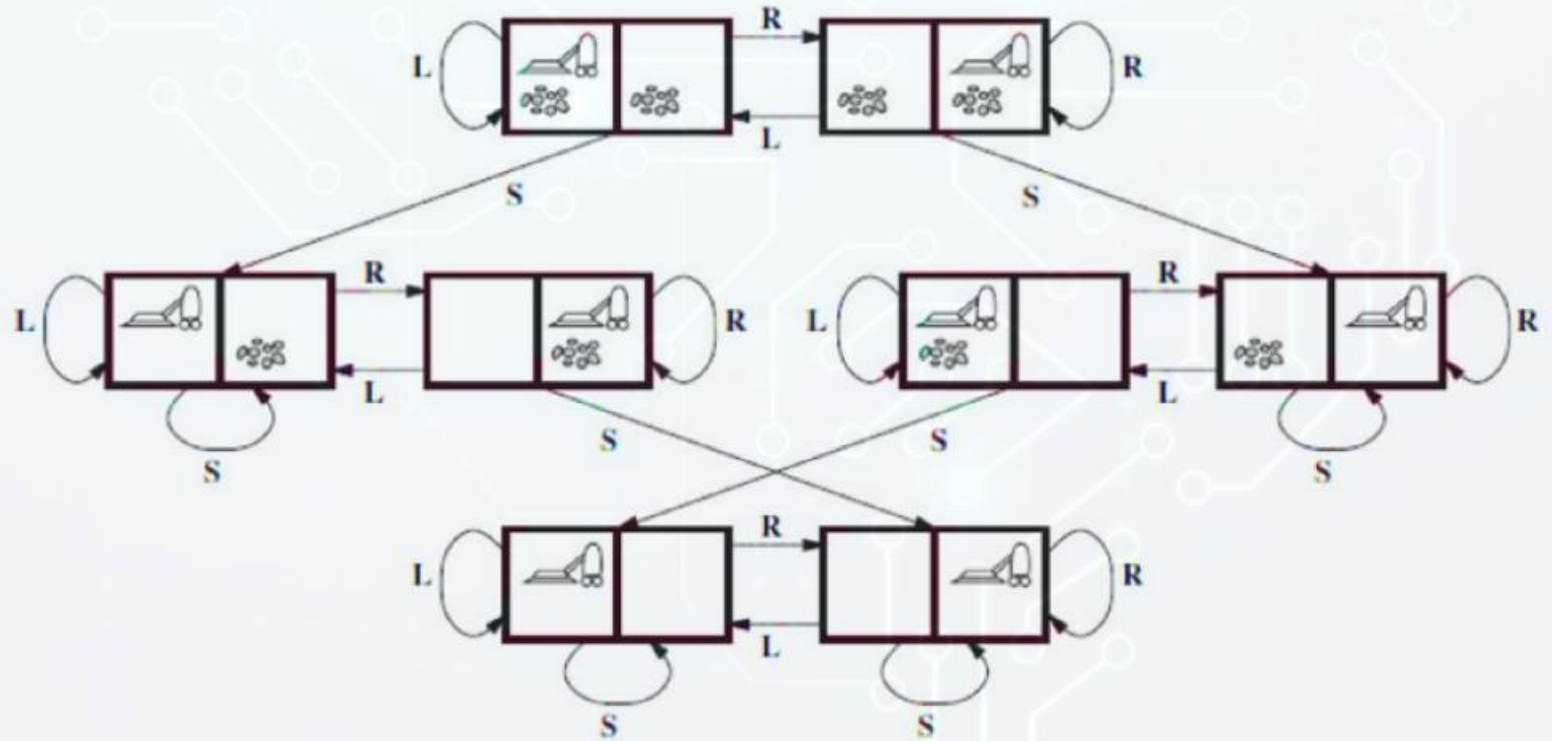
- Robot aspirateur

L: Left.

R: Right.

S: Suck

- A est propre, B est propre : (P, P)
- A est propre, B est sale : (P, S)
- A est sale, B est propre : (S, P)
- A est sale, B est sale : (S, S)



Exemples de problèmes

- **Robot aspirateur**

- **Etat** : La position du robot (A ou B). L'état de propreté des deux emplacements (sale ou propre).
→ Il y a donc **8 états possibles** (2 positions $\times 2^2$ configurations de propreté).

- **État initial** : N'importe quel état.

- **Actions** : Trois Actions L: Left, R: Right, S: Suck

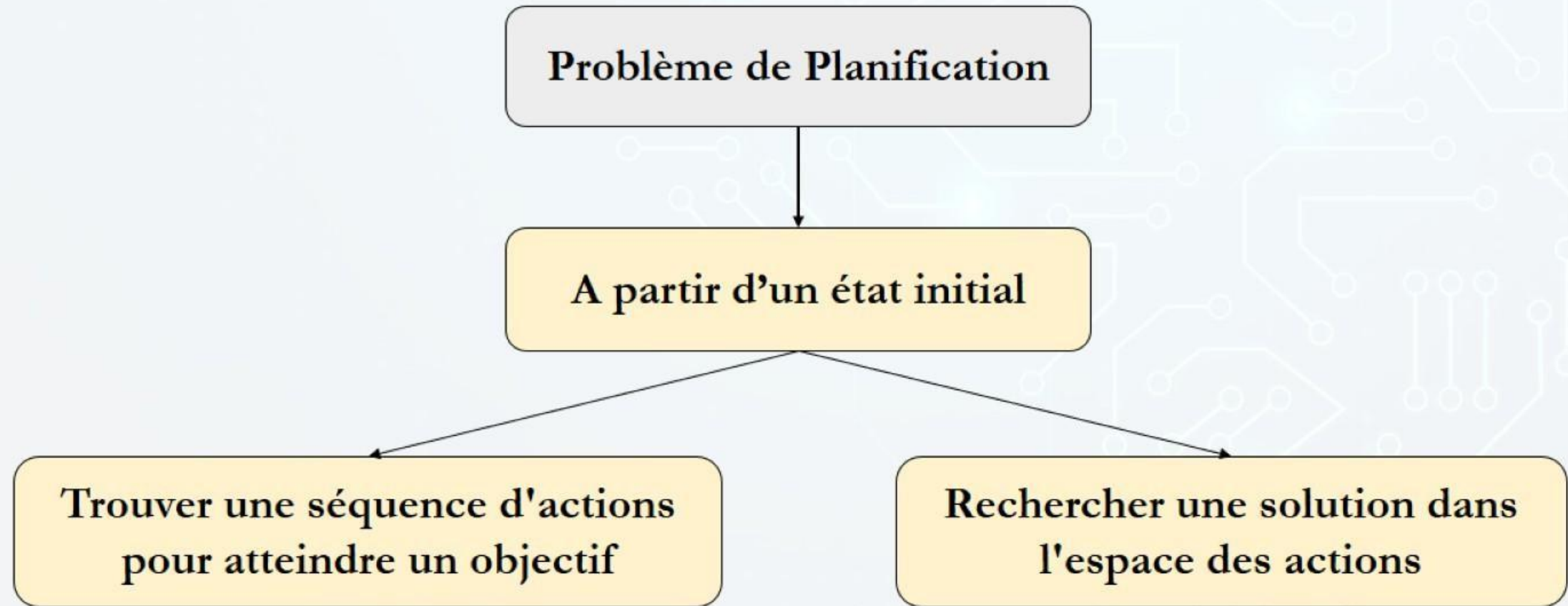
- **Fonction de successeur** : Les actions ont un effet attendu sauf si elles sont inutiles.

- **Test de but** : Emplacements propres.

- **Coûts des actions** : Chaque action a un coût de 1.

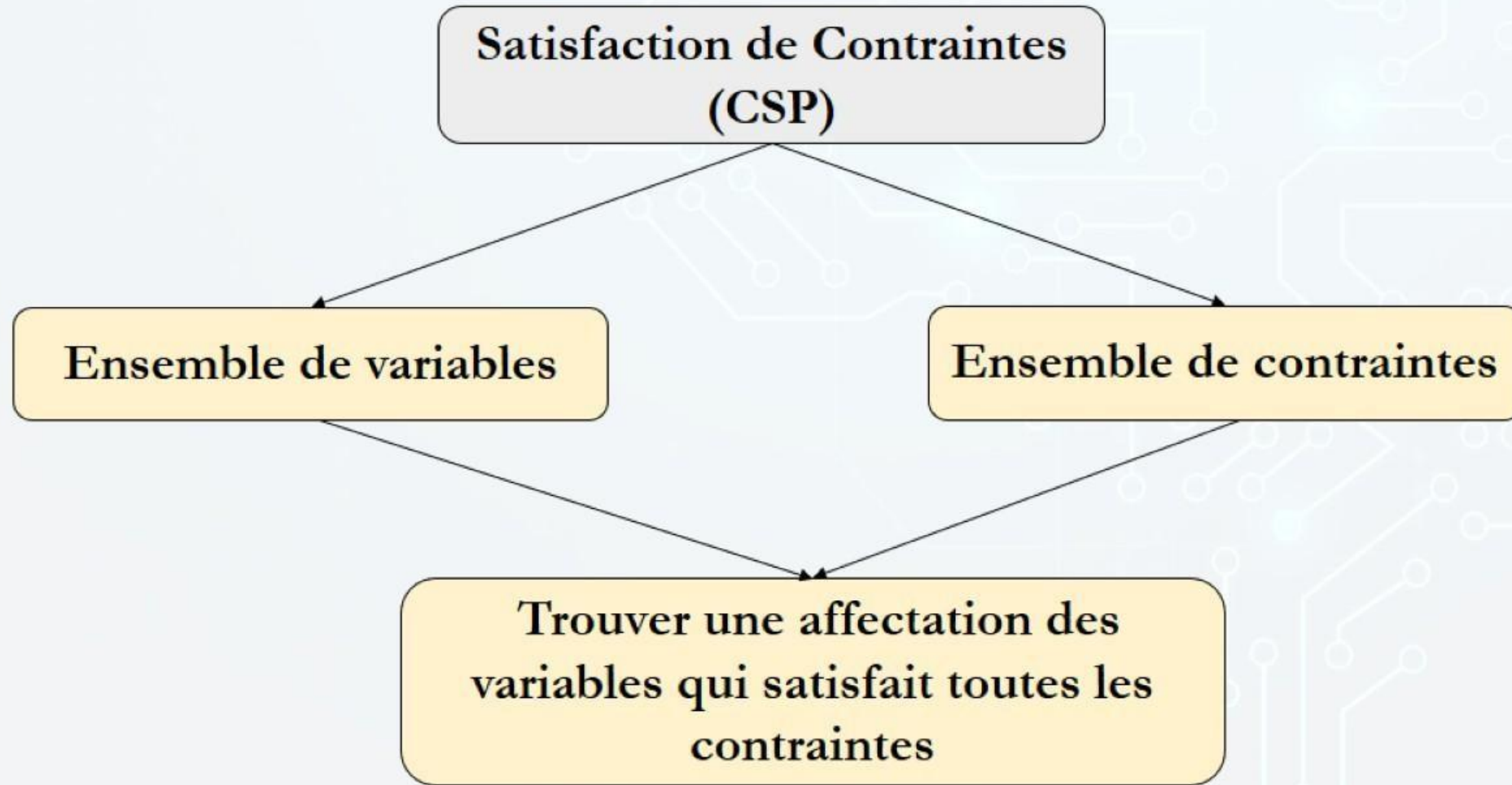
Le coût total correspond au nombre d'états visités jusqu'à atteindre l'objectif.

Systeme de Résolution de Problème



* Il existe une notion de **temps** ou d'**ordre d'exécution** des actions.

Systeme de Résolution de Problème



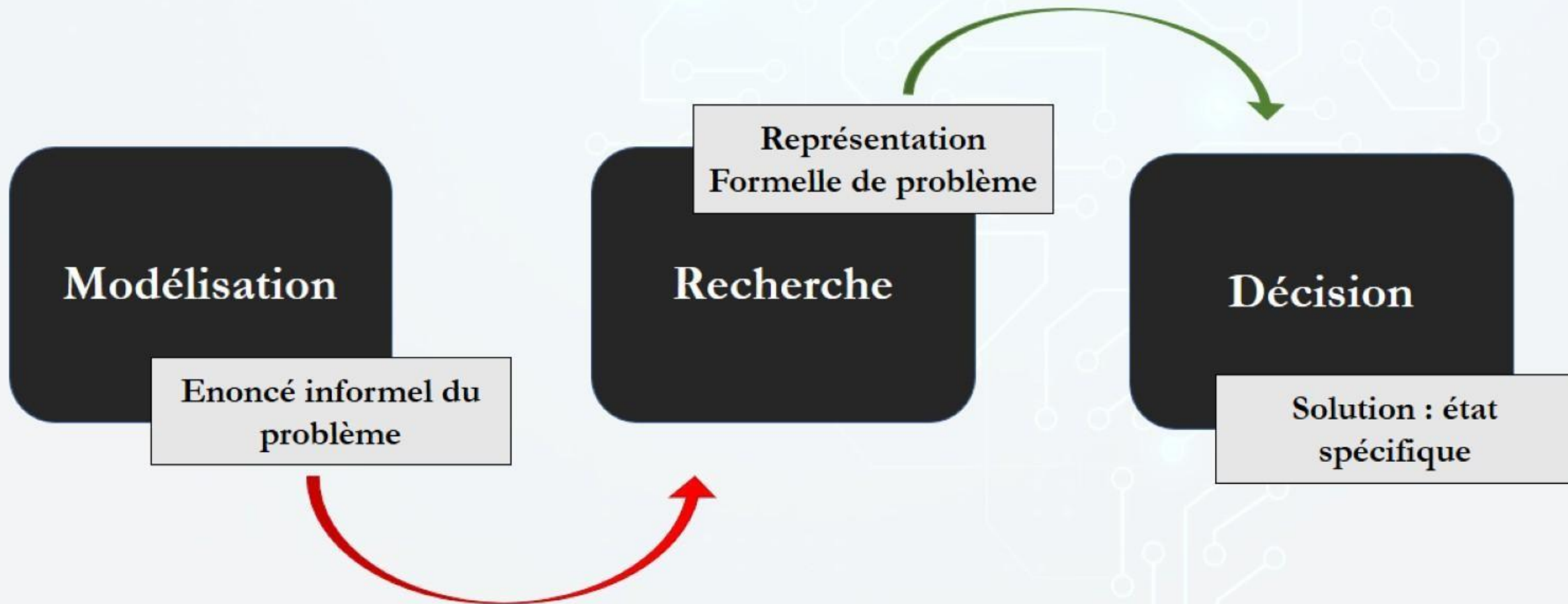
* Il n'y a pas de notion d'actions ni de temps

Systeme de Résolution de Problème

Les concepts caractérisant le Système de Résolution de Problèmes :

- Espace de Recherche
- Ensemble de règles de transformation/production
- Stratégie de Recherche
- Connaissance supplémentaire (Heuristique)

Représentation Formelle



Pour un même problème, **plusieurs représentations** par graphe d'états sont possibles.

Stratégies de recherche

- Pour ces problèmes, les algorithmes de recherche constituent une technique générale de résolution.
- Ces algorithmes guident la recherche d'une solution en sélectionnant les états à développer.
- Ils permettent également de gérer le retour sur les choix effectués.
- Leur objectif est d'optimiser la recherche tout en évitant l'explosion combinatoire.

Algorithme de résolution de problème simple

- Les stratégies de recherche indiquent à l'algorithme quel état développer en premier.
- L'objectif d'une stratégie est d'organiser l'exploration de l'espace d'états d'un problème.
- **Définition de la fonction de recherche :**
 1. Initialiser Struct avec le nœud représentant l'état initial du problème (racine de l'arbre).
 2. Tant qu'il reste des nœuds dans Struct :
 - Choisir un nœud à développer en suivant la stratégie définie.
 - Si ce nœud correspond à un état but, retourner la solution.
 - Sinon, développer le nœud en ajoutant ses successeurs à Struct.
 3. Si aucun état but n'est trouvé, retourner **Échec**.

Algorithme de résolution de problème simple

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action  
  persistent: seq, an action sequence, initially empty  
               state, some description of the current world state  
               goal, a goal, initially null  
               problem, a problem formulation  
  
  state  $\leftarrow$  UPDATE-STATE(state, percept)  
  if seq is empty then  
    goal  $\leftarrow$  FORMULATE-GOAL(state)  
    problem  $\leftarrow$  FORMULATE-PROBLEM(state, goal)  
    seq  $\leftarrow$  SEARCH(problem)  
    if seq = failure then return a null action  
  action  $\leftarrow$  FIRST(seq)  
  seq  $\leftarrow$  REST(seq)  
  return action
```


Algorithme de résolution de problème simple

```
fonction RECHERCHE(état_initial, SUCCESSEURS, TEST_BUT, COÛT)
  noeuds_à_traiter ← CRÉER_LISTE(CRÉER_NOEUD(état_initial, [], 0))
  boucle
    si VIDE ?(noeuds_à_traiter) alors renvoyer échec
    noeud ← ENLEVER_PREMIER_NOEUD(noeuds_à_traiter)
    si TEST_BUT(ÉTAT(noeud)) = vrai
      alors renvoyer CHEMIN(noeud), ÉTAT(noeud)
    pour tout (action, état) dans SUCCESSEURS(ÉTAT(noeud))
      chemin ← [action, CHEMIN(noeud)]
      coût_du_chemin ← COÛT_DU_CHEMIN(noeud) + COÛT(état)
      s ← créer_noeud(état, chemin, coût_du_chemin)
      INSÉRER(s, noeuds_à_traiter)
```

Evaluation d'algorithme de Recherche

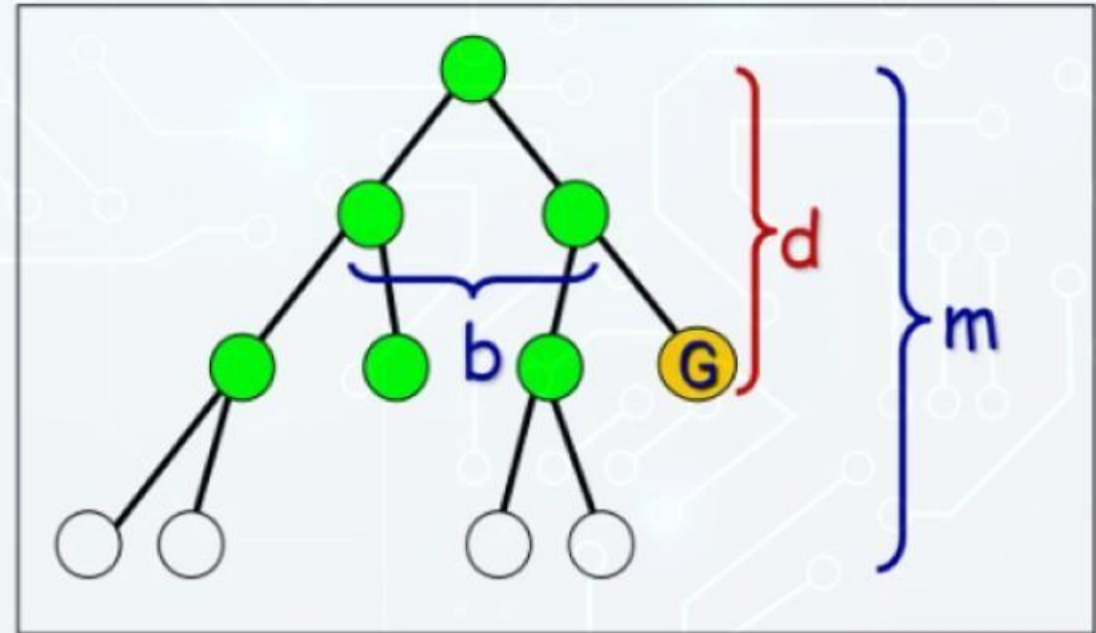
Les critères d'évaluation sont :

- ❑ **Complétude** : La méthode garantit de trouver une solution si elle existe.
- ❑ **Terminaison (décidabilité)** : La méthode s'arrête, même si la solution n'existe pas, et signale un échec si l'espace d'états est fini.
- ❑ **Complexité en temps** : Mesure de l'efficacité algorithmique en fonction du nombre d'états développés.
- ❑ **Complexité en espace** : Réfère à la quantité de mémoire utilisée pendant l'exécution.
- ❑ **Optimalité (admissibilité)** : Évalue l'optimalité de la solution trouvée par rapport à celles qui existent, en se basant sur une fonction de coût.

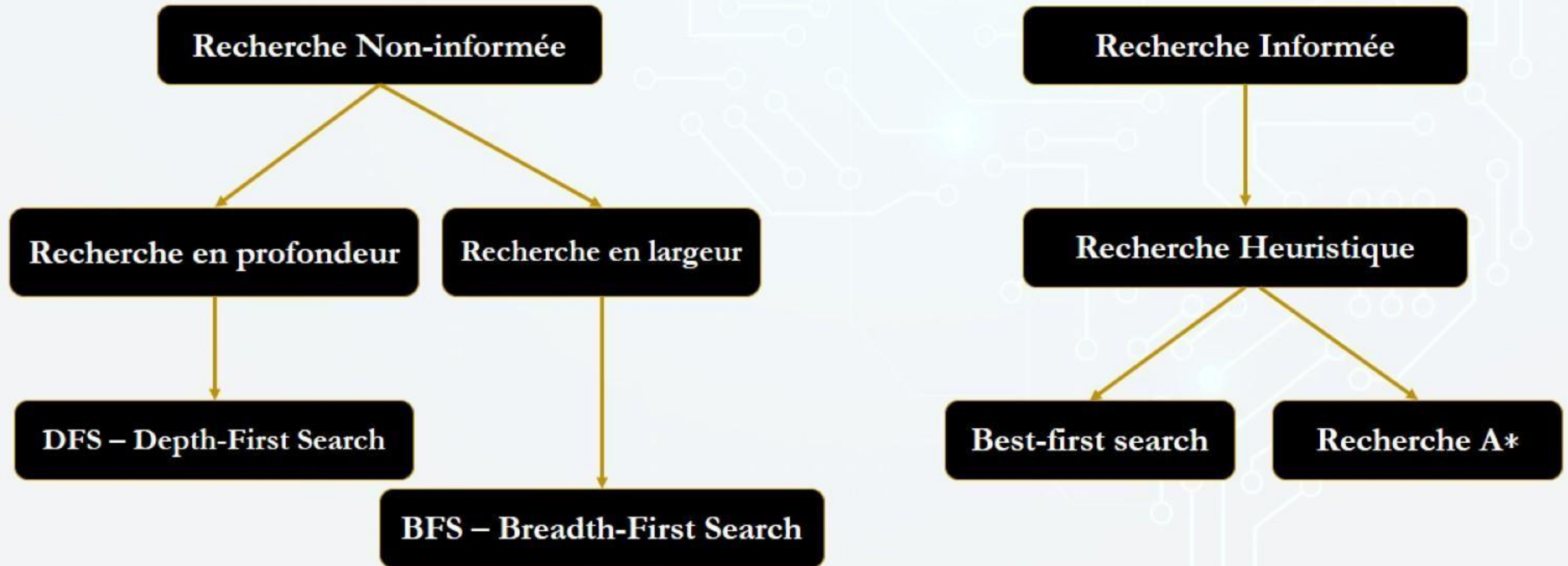
Evaluation d'algorithme de Recherche

La complexité en temps dépend des paramètres suivants :

- **b** : Facteur de branchement maximal de l'arbre de recherche, représentant le nombre maximal de successeurs pour un état.
- **d** : Profondeur du nœud solution, c'est-à-dire le nombre d'actions nécessaires pour atteindre la solution depuis l'état initial.
- **m** : Souvent être infinie dans certains problèmes. Profondeur maximale de l'arbre de recherche, représentant la taille de l'espace d'états



Algorithmes de Recherche



Recherche non-informée

- Les algorithmes de recherche non informés ne disposent d'aucune information supplémentaire pour choisir les meilleurs chemins.
- Contrairement aux programmes d'échecs qui sélectionnent les coups les plus prometteurs, ces algorithmes explorent toutes les possibilités sans distinction.
- Ils fonctionnent en testant systématiquement tous les chemins possibles depuis l'état initial.

Recherche non-informée

❑ Recherche en profondeur d'abord / DFS – Depth-First Search

- Explore un chemin **au maximum** avant de revenir en arrière.
- Si une solution est trouvée → Arrêt
- Si la solution n'est trouvée : état d'échec → Poursuivre la recherche (Backtracking).
- Définir une profondeur maximale.
- Complexité liée à l'ordre d'exploration des branches

Recherche non-informée

❑ Recherche en profondeur d'abord / DFS – Depth-First Search

Représentation : Pile (LIFO)

Empiler le nœud correspondant à l'état initial (racine de l'arbre) dans P

Tant que P est non vide **faire**

Dépiler le sommet N de P

Si N correspond à un état but **alors**

Sortir de l'algorithme avec cette solution

Sinon

Générer la liste L des successeurs de N

Pour chaque élément S de L (de droite à gauche !) **faire**

Empiler S (Rq : le plus à gauche sera en sommet)

Fin de Tant que

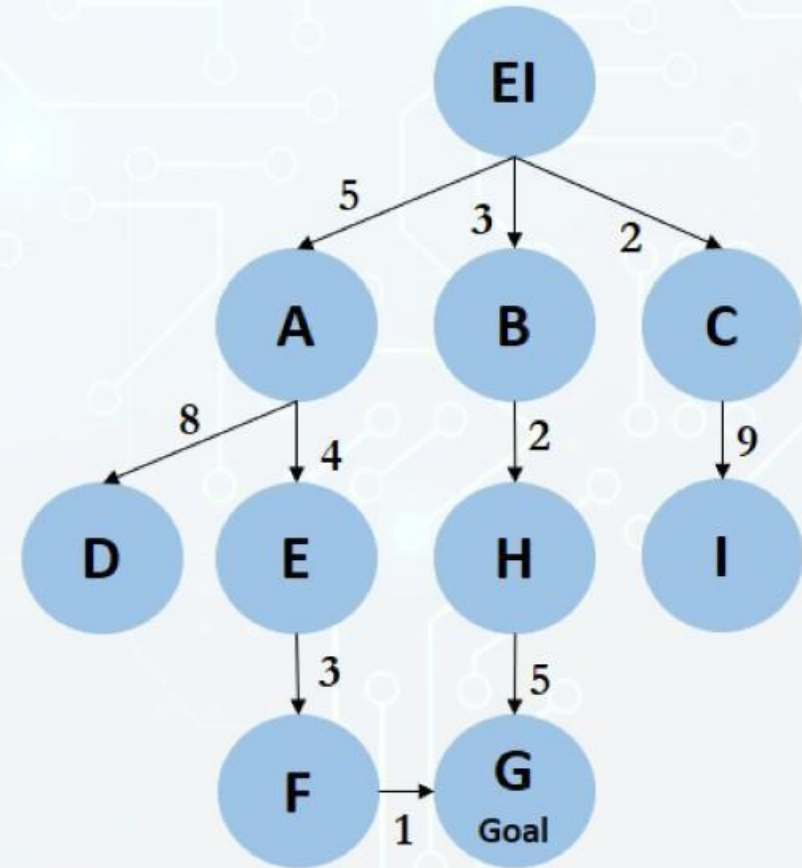
Retourner Echec

Recherche non-informée

❑ Recherche en profondeur d'abord / DFS – Depth-First Search

expand.node	Nodes list
	{E1}

- NB nodes tested : 0
- NB expanded : 0

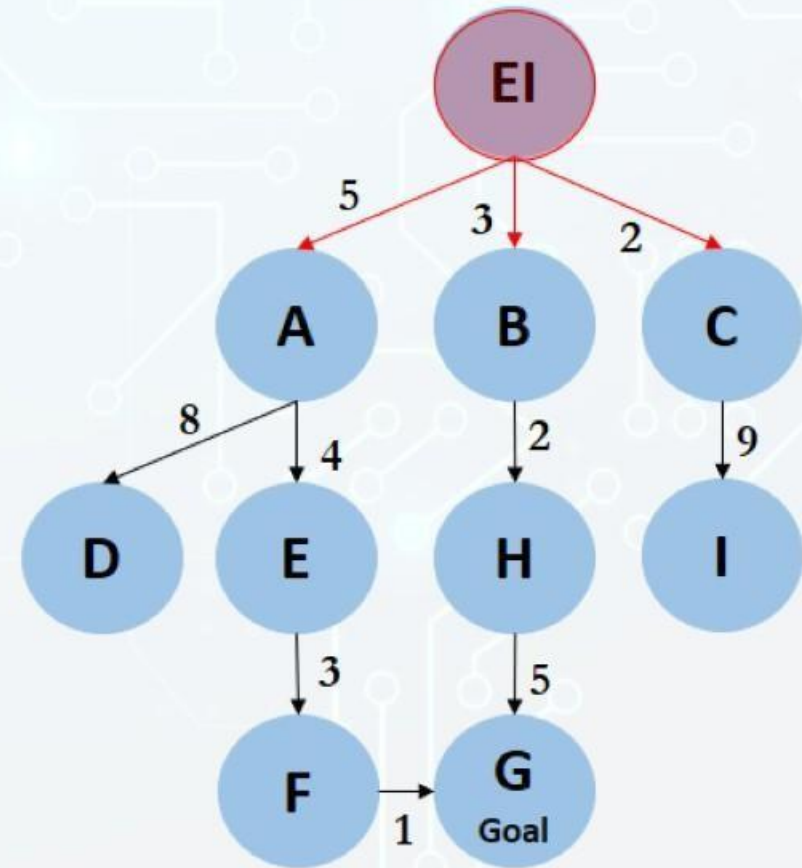


Recherche non-informée

❑ Recherche en profondeur d'abord / DFS – Depth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{C,B,A}

- NB nodes tested : 1
- NB expanded : 1

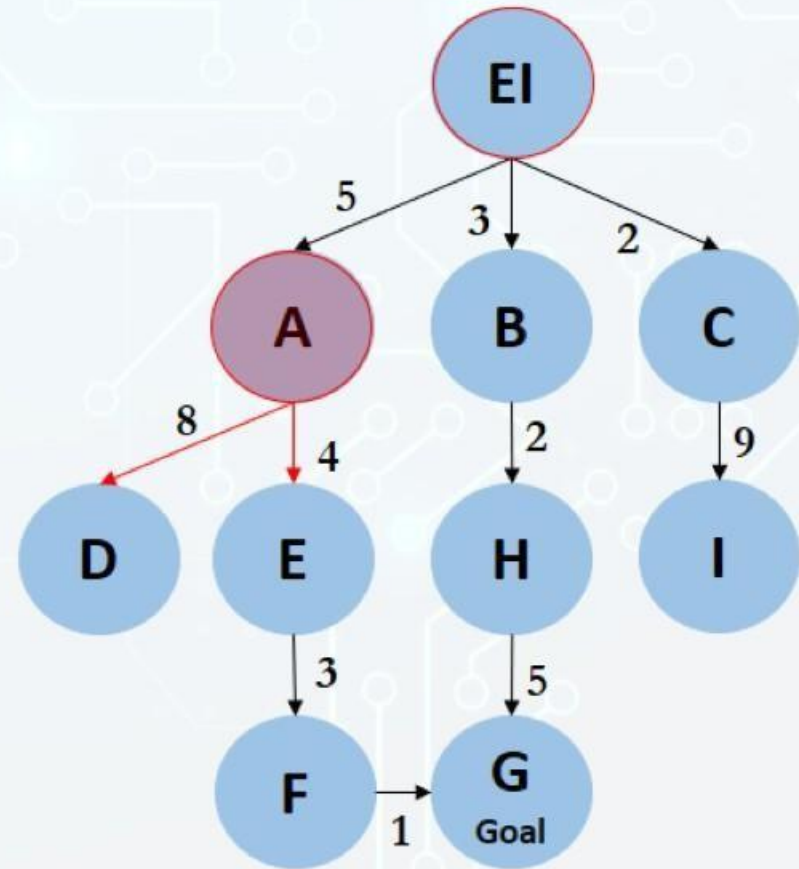


Recherche non-informée

❑ Recherche en profondeur d'abord / DFS – Depth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{C,B,A}
A not Goal	{C,B,E,D}

- NB nodes tested : 2
- NB expanded : 2

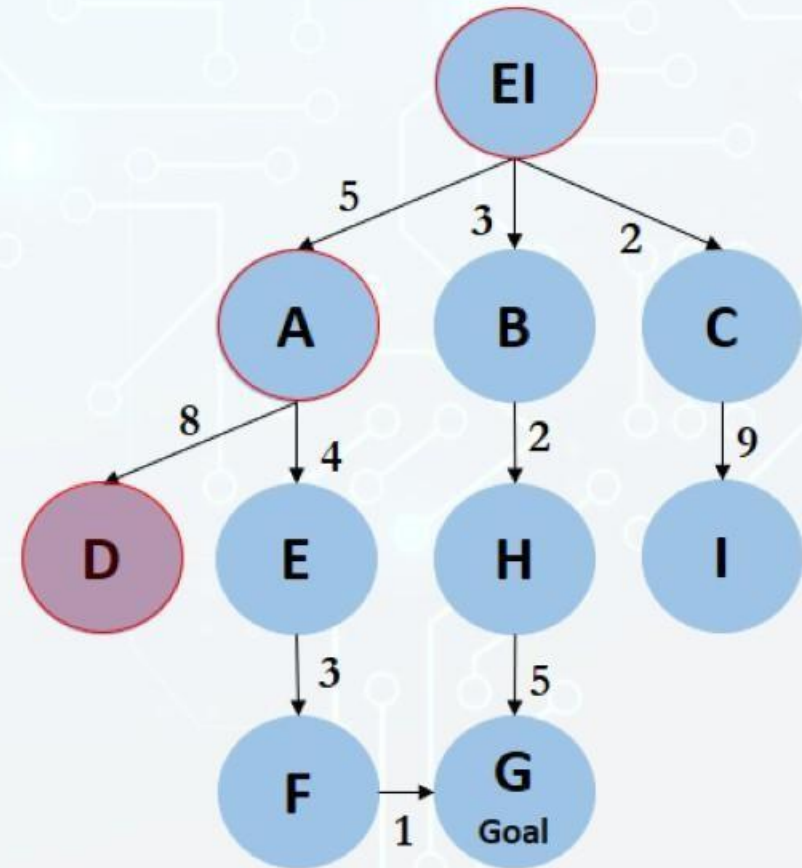


Recherche non-informée

❑ Recherche en profondeur d'abord / DFS – Depth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{C,B,A}
A not Goal	{C,B,E,D}
D not Goal	{C,B,E}

- NB nodes tested : 3
- NB expanded : 2

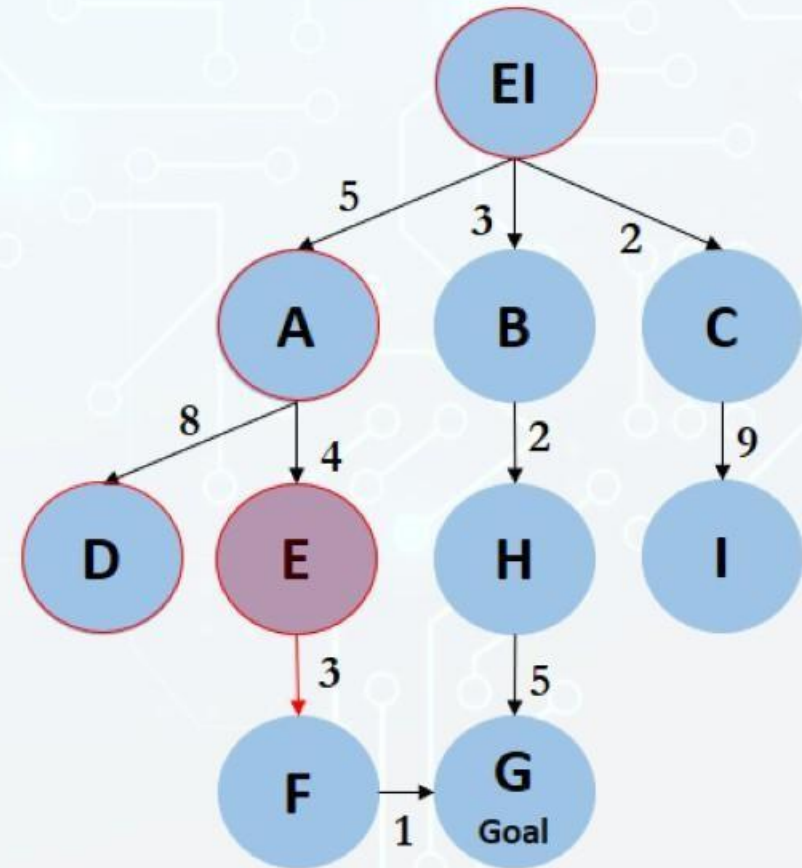


Recherche non-informée

❑ Recherche en profondeur d'abord / DFS – Depth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{C,B,A}
A not Goal	{C,B,E,D}
D not Goal	{C,B,E}
E not Goal	{C,B,F}

- NB nodes tested : 4
- NB expanded : 3

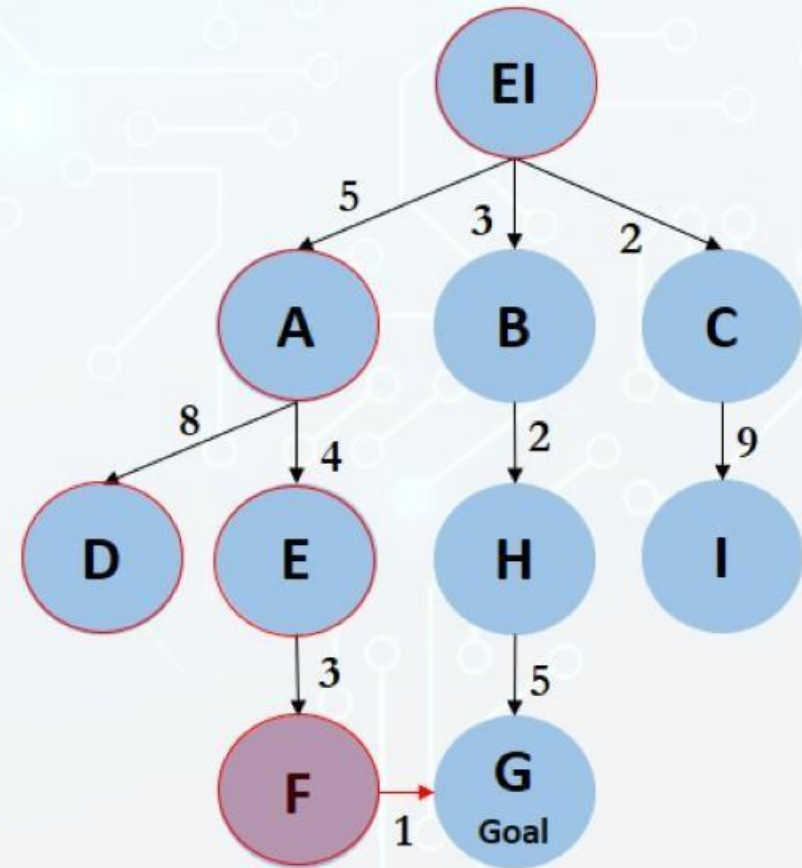


Recherche non-informée

❑ Recherche en profondeur d'abord / DFS – Depth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{C,B,A}
A not Goal	{C,B,E,D}
D not Goal	{C,B,E}
E not Goal	{C,B,F}
F not Goal	{C,B,G}

- NB nodes tested : 5
- NB expanded : 4

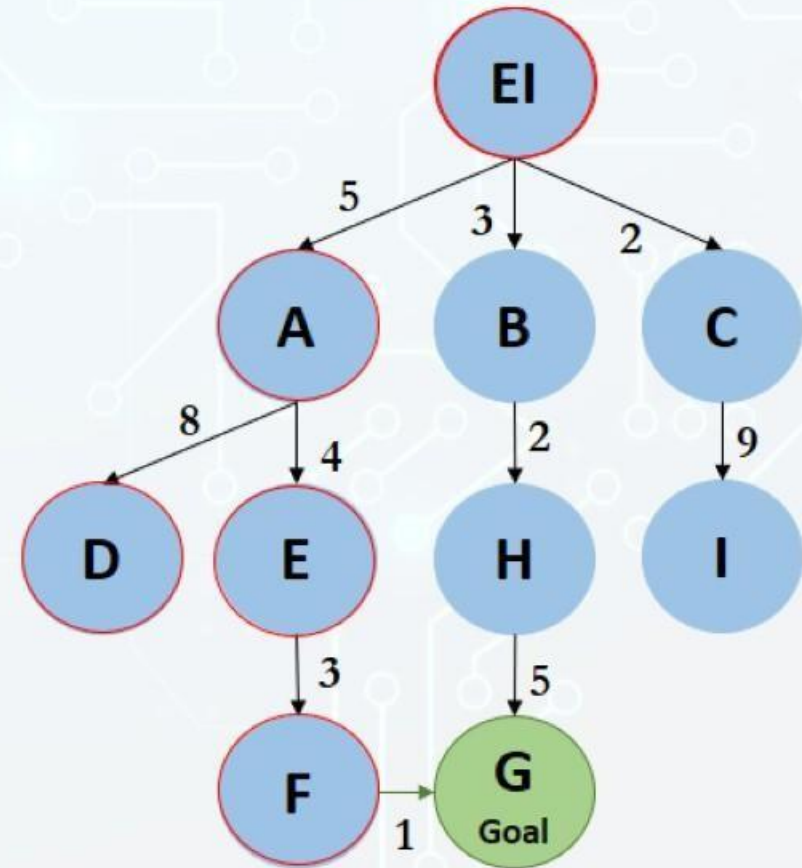


Recherche non-informée

❑ Recherche en profondeur d'abord / DFS – Depth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{C,B,A}
A not Goal	{C,B, E ,D}
D not Goal	{C,B,E}
E not Goal	{C,B, F }
F not Goal	{C,B, G }
G the Goal	

- NB nodes tested : 6
- NB expanded : 4



Recherche non-informée

❑ Recherche en profondeur d'abord / DFS – Depth-First Search

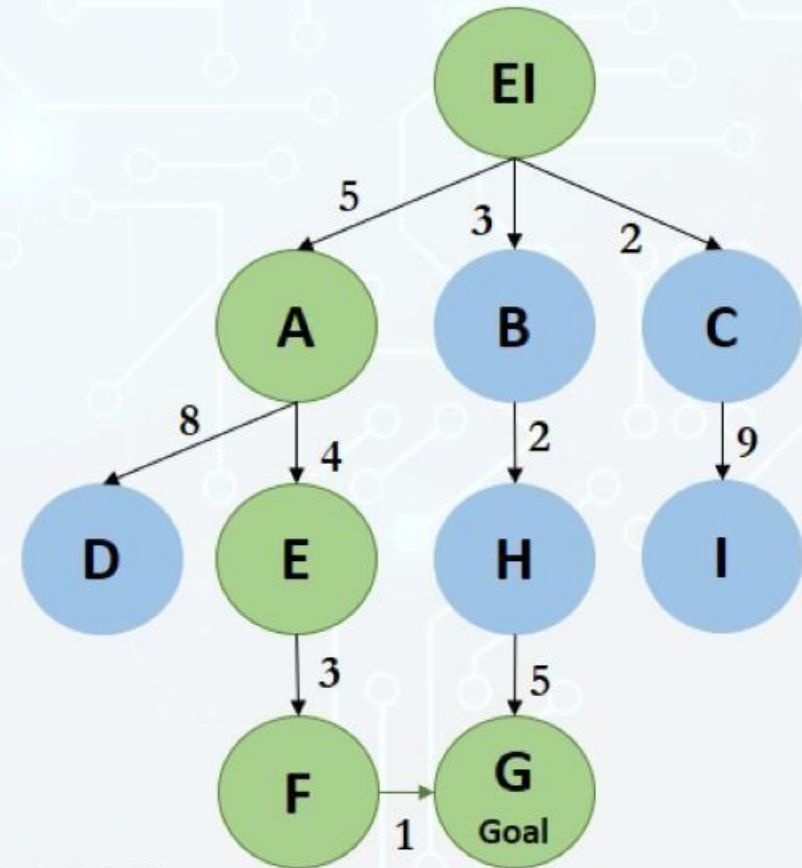
expand.node	Nodes list
	{E1}
E1 not Goal	{C,B,A}
A not Goal	{C,B, E,D }
D not Goal	{C,B,E}
E not Goal	{C,B, F }
F not Goal	{C,B, G }
G the Goal	{C,B} No expand

- NB nodes tested : 6

- NB expanded : 4

- Chemin : {E1,A,E,F,G}

- Coût : 13



Recherche non-informée

❑ Recherche en profondeur d'abord / DFS – Depth-First Search

▪ Propriétés

- **Complétude** : Incomplet si l'espace d'états est infini → DFS peut revisiter des nœuds déjà explorés, entraînant des boucles et une exploration inefficace. **Solution** : Ajouter une liste des nœuds visités pour éviter les répétitions. **Sinon il est Complet** même si aucune solution n'est trouvée, l'algorithme s'arrête et signale un échec.
- Sensibilité à l'ordre des successeurs.
- Utilise moins de mémoire que BFS et performant dans un environnement de mémoire limitée.
- **Complexité temps** : Le nombre de nœuds développés est de l'ordre de b^m ($O(b^m)$) avec b le facteur de branchement et m la profondeur maximale de l'arbre.
- **Complexité en espace** : elle est linéaire de l'ordre de $b \cdot m$ ($O(b \cdot m)$).
- **Optimalité** :
Il peut explorer des chemins longs avant d'atteindre la solution optimale.
Ne garantit pas de trouver le chemin le plus court dans un graphe pondéré.

Recherche non-informée

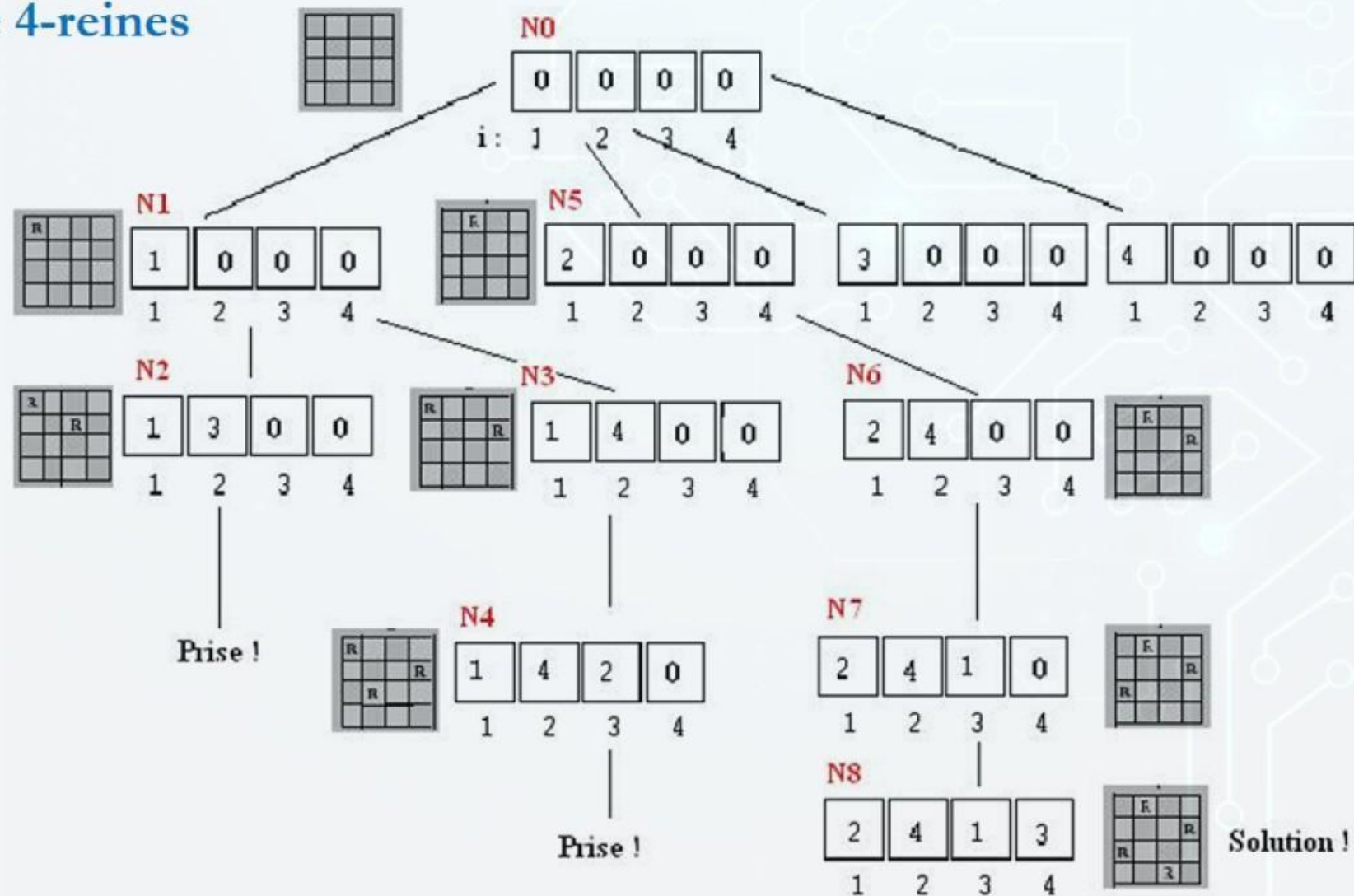
❑ Recherche en profondeur d'abord / DFS – Depth-First Search

▪ Exercice

- Donner une solution pour le problème de 4-reines avec la stratégie en profondeur d'abord.

Recherche non-informée

Problème de 4-reines



Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

- Explore tous les nœuds d'un niveau avant de passer au suivant.
- Continuer niveau par niveau jusqu'à arriver au but.
- La recherche s'arrête lorsqu'un état but est trouvé ou une profondeur maximale est trouvée.

Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

Représentation : File

Insérer le nœud correspondant à l'état initial (racine de l'arbre) dans F

Tant que F est non vide **faire**

Extraire la tête (le nœud le plus ancien) N de F

Si N correspond à un état but **alors**

Sortir de l'algorithme avec cette solution

 Sinon

Générer la liste L des successeurs de N

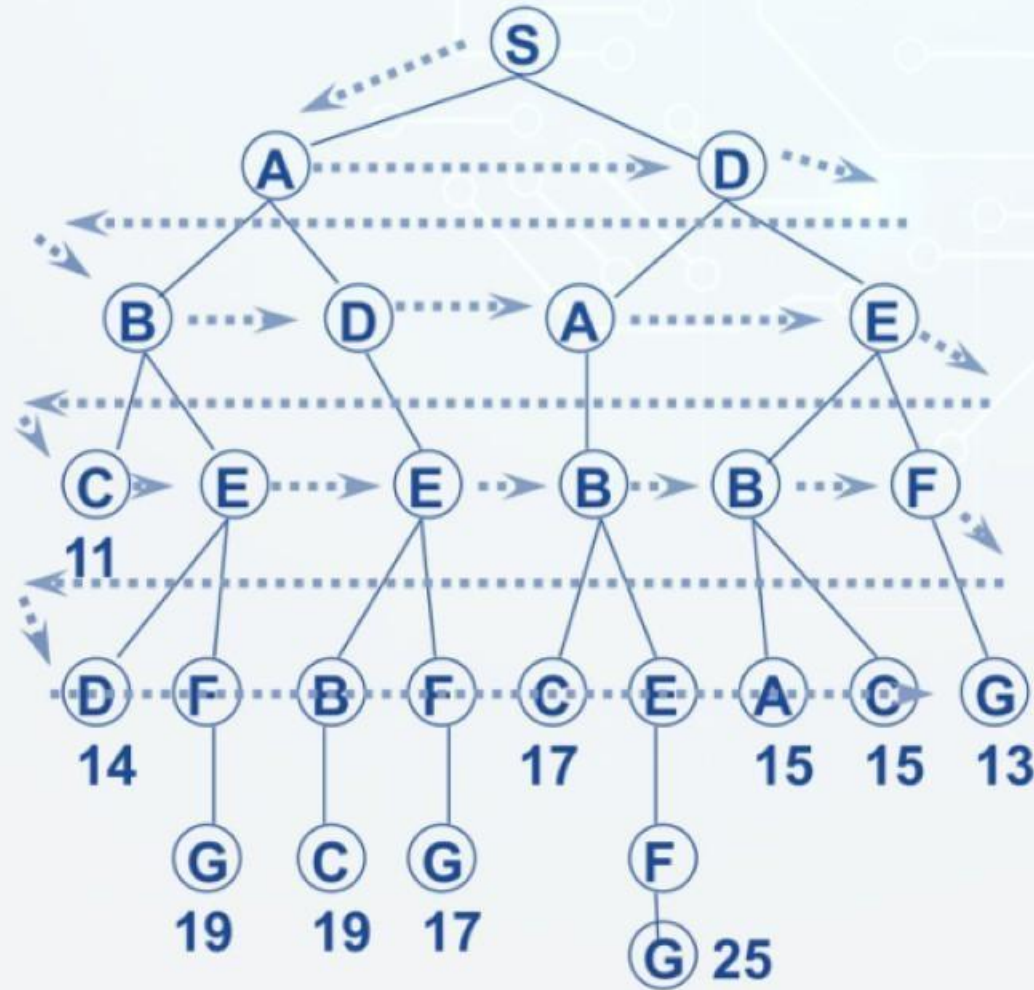
Pour chaque élément S de L (de gauche à droite !) **faire Insérer** S dans F(en fin de la file)

Fin de Tant que

Retourner Echec

Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

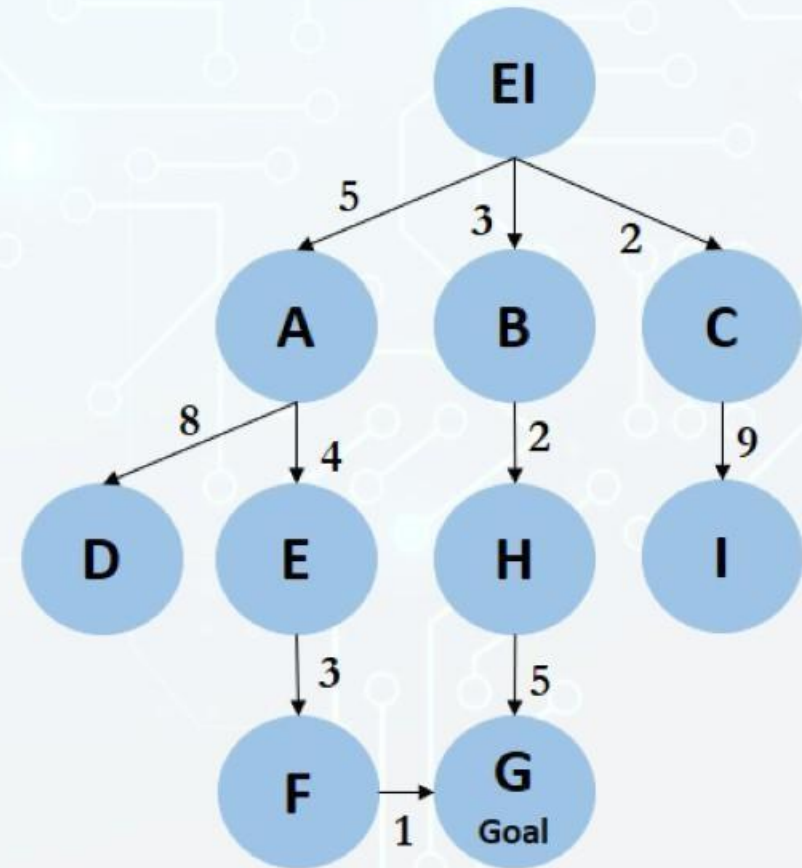


Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

expand.node	Nodes list
	{E1}

- NB nodes tested : 0
- NB expanded : 0

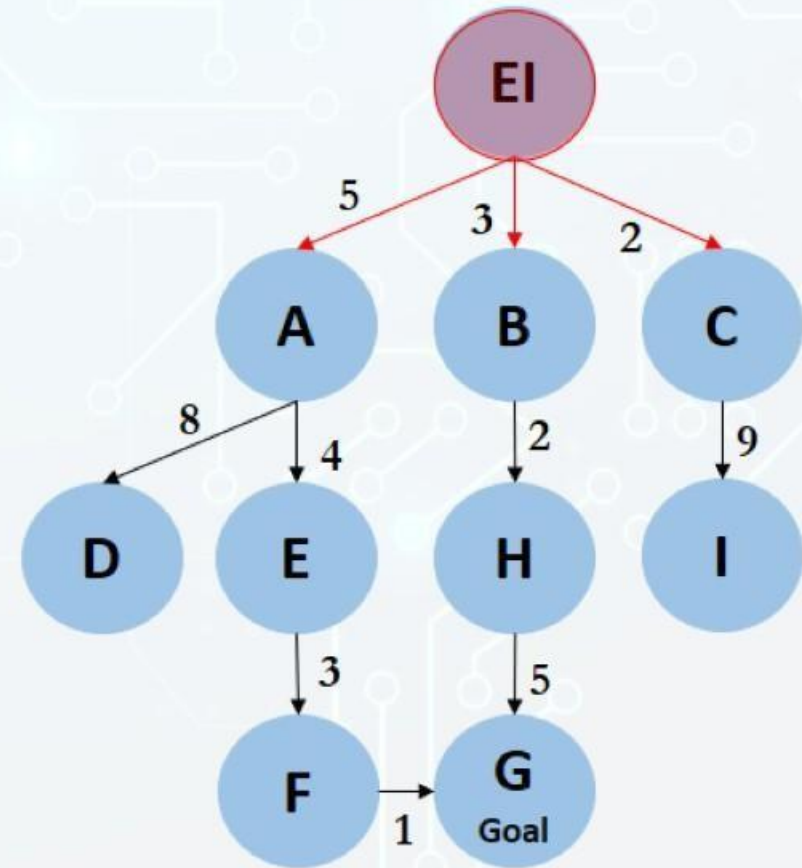


Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{A,B,C}

- NB nodes tested : 1
- NB expanded : 1

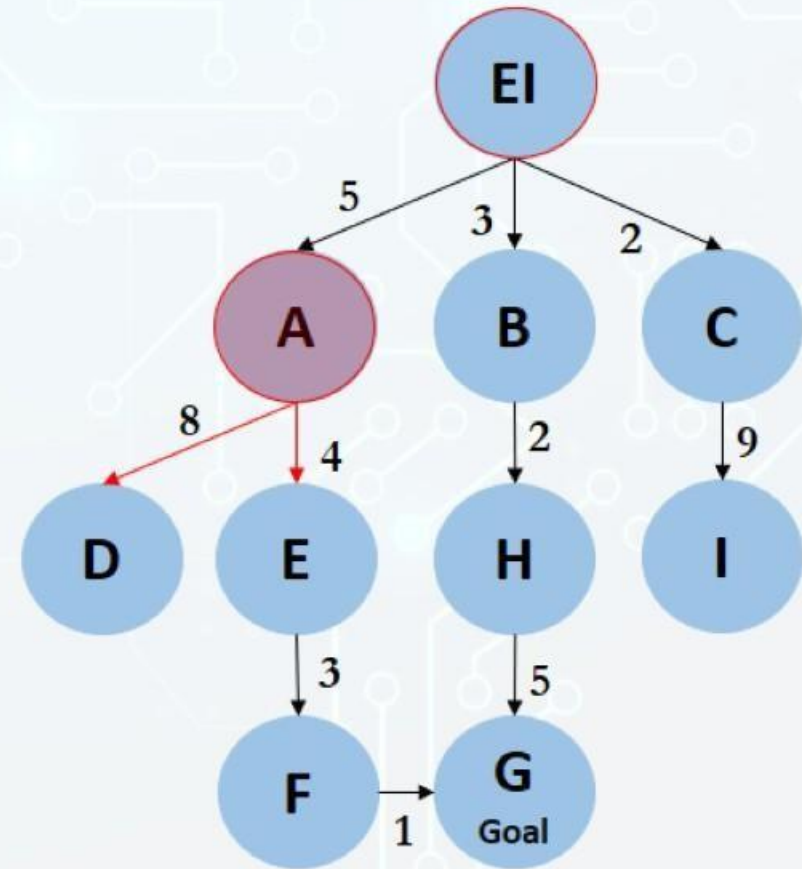


Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{A,B,C}
A not Goal	{B,C,D,E}

- NB nodes tested : 2
- NB expanded : 2

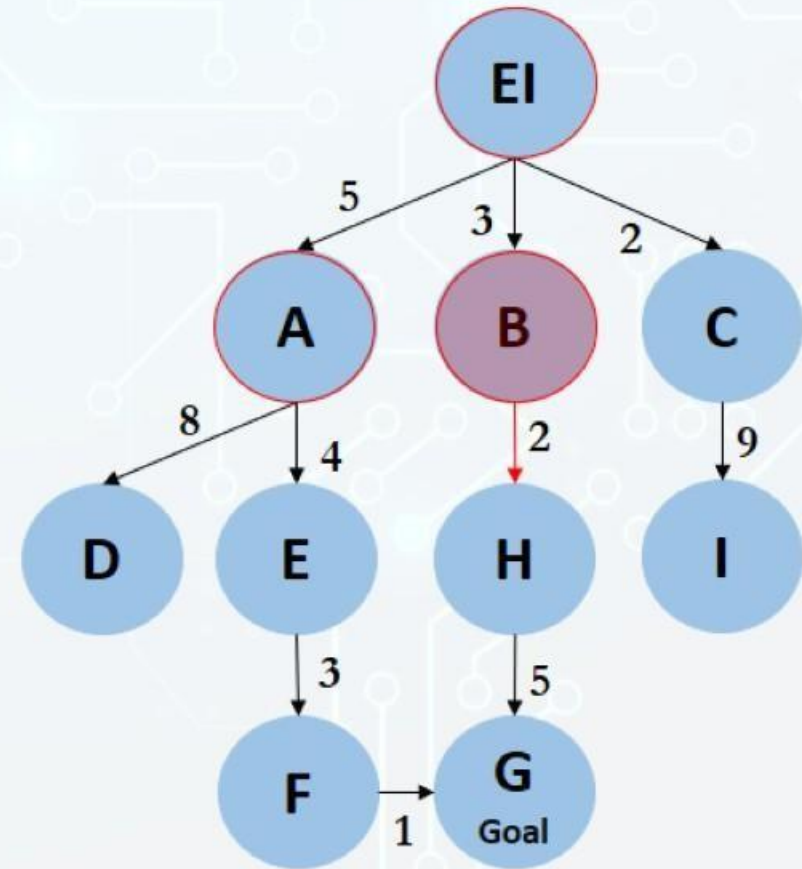


Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{A,B,C}
A not Goal	{B,C,D,E}
B not Goal	{C,D,E,H}

- NB nodes tested : 3
- NB expanded : 3

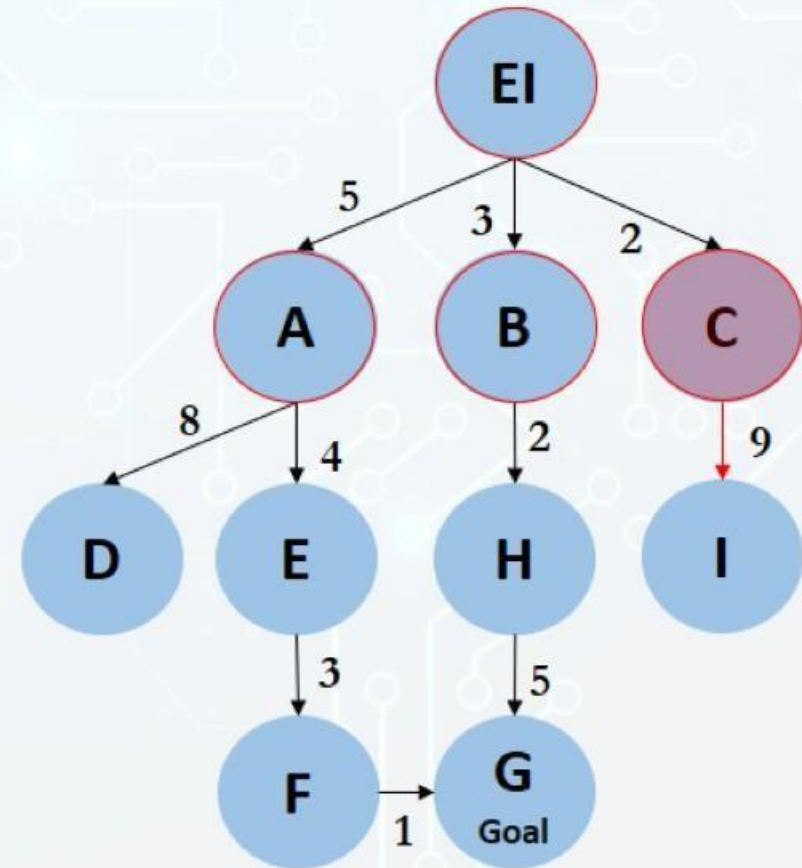


Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{A,B,C}
A not Goal	{B,C,D,E}
B not Goal	{C,D,E,H}
C not Goal	{D,E,H,I}

- NB nodes tested : 4
- NB expanded : 4

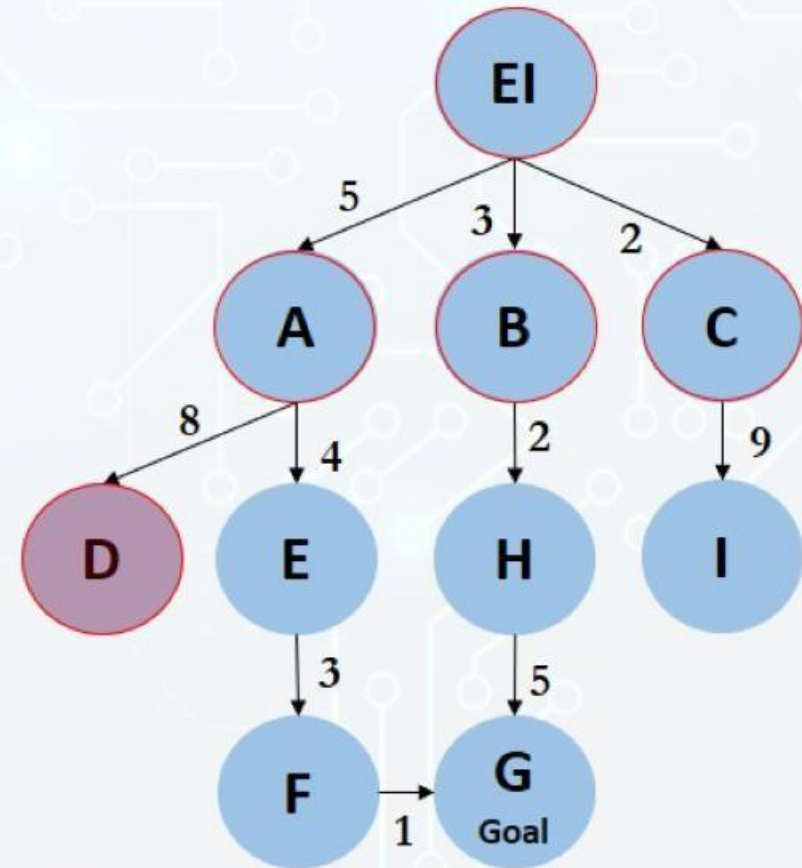


Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{A,B,C}
A not Goal	{B,C,D,E}
B not Goal	{C,D,E,H}
C not Goal	{D,E,H,I}
D not Goal	{E,H,I}

- NB nodes tested : 5
- NB expanded : 4

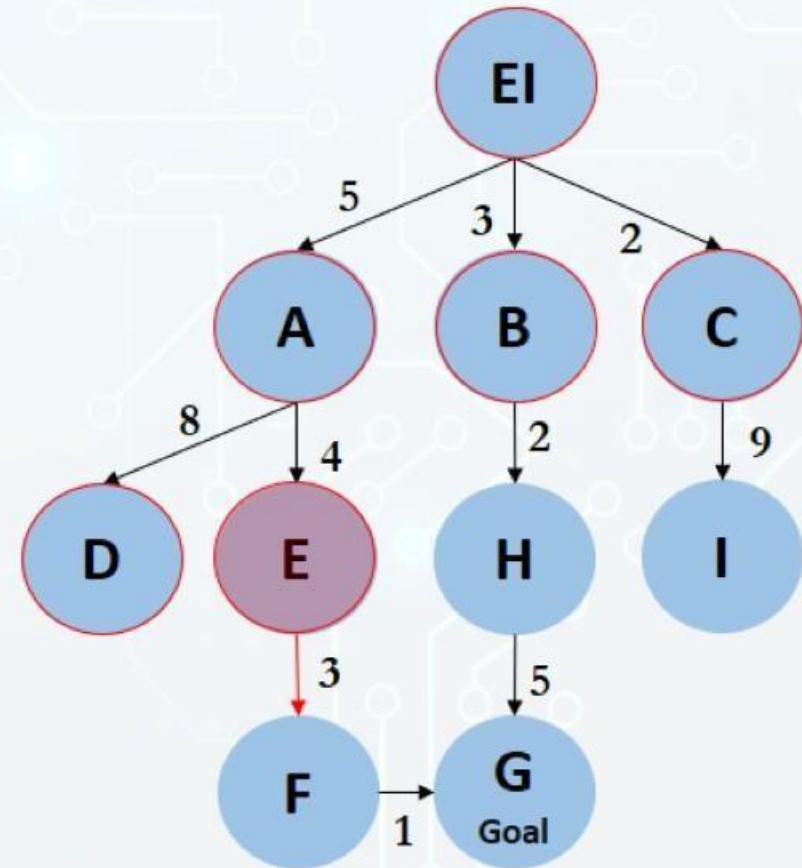


Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{A,B,C}
A not Goal	{B,C,D,E}
B not Goal	{C,D,E,H}
C not Goal	{D,E,H,I}
D not Goal	{E,H,I}
E not Goal	{H,I,F}

- NB nodes tested : 6
- NB expanded : 5

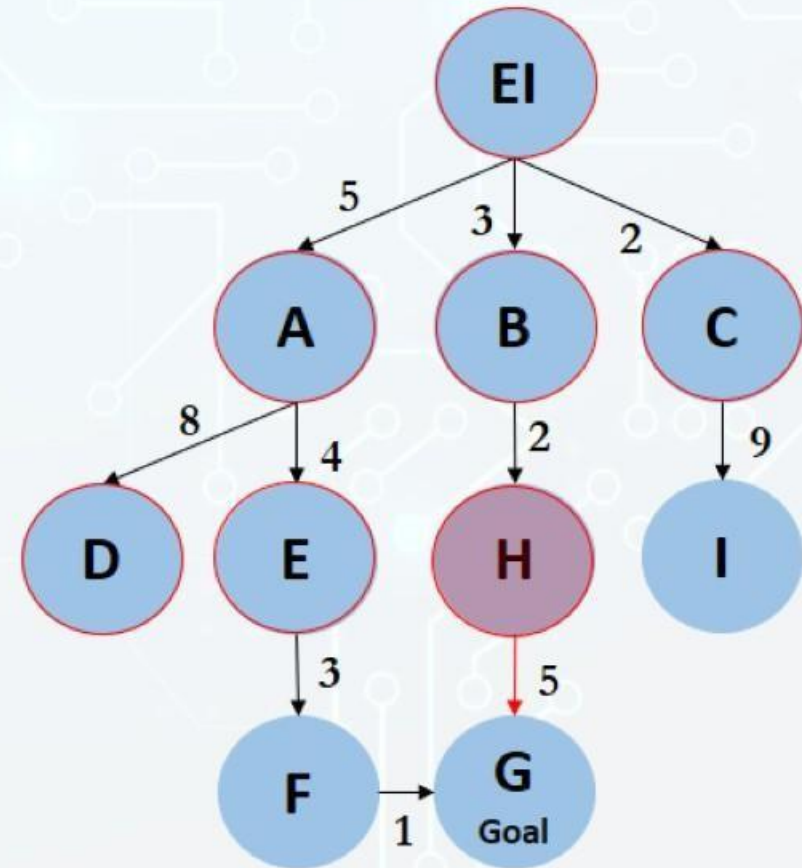


Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{A,B,C}
A not Goal	{B,C,D,E}
B not Goal	{C,D,E,H}
C not Goal	{D,E,H,I}
D not Goal	{E,H,I}
E not Goal	{H,I,F}
H not Goal	{I,F,G}

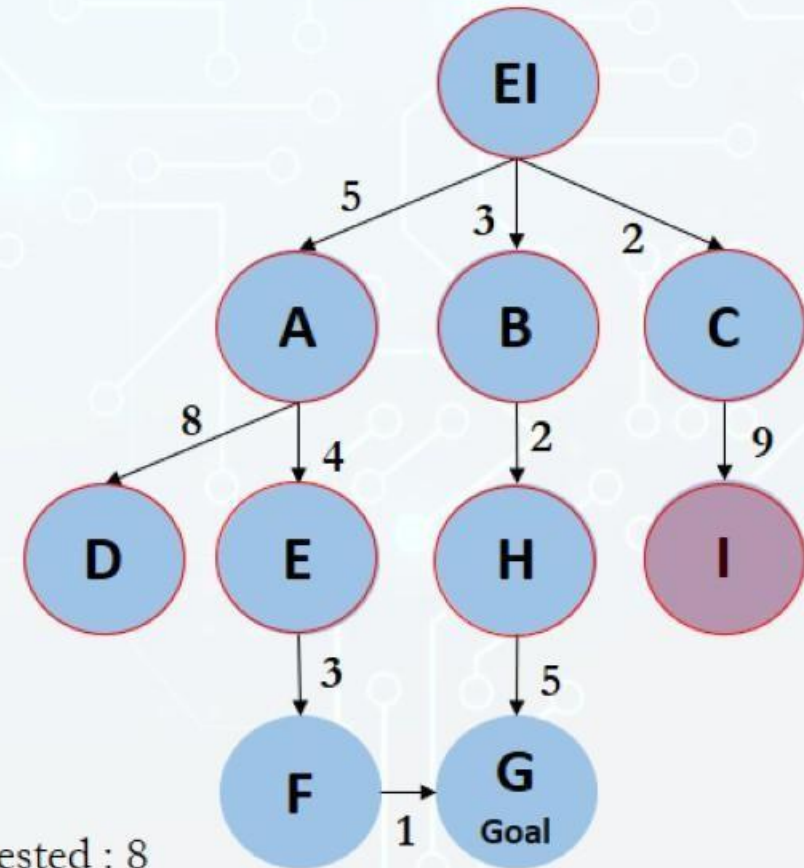
- NB nodes tested : 7
- NB expanded : 6



Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{A,B,C}
A not Goal	{B,C,D,E}
B not Goal	{C,D,E,H}
C not Goal	{D,E,H,I}
D not Goal	{E,H,I}
E not Goal	{H,I,F}
H not Goal	{I,F,G}
I not Goal	{F,G}

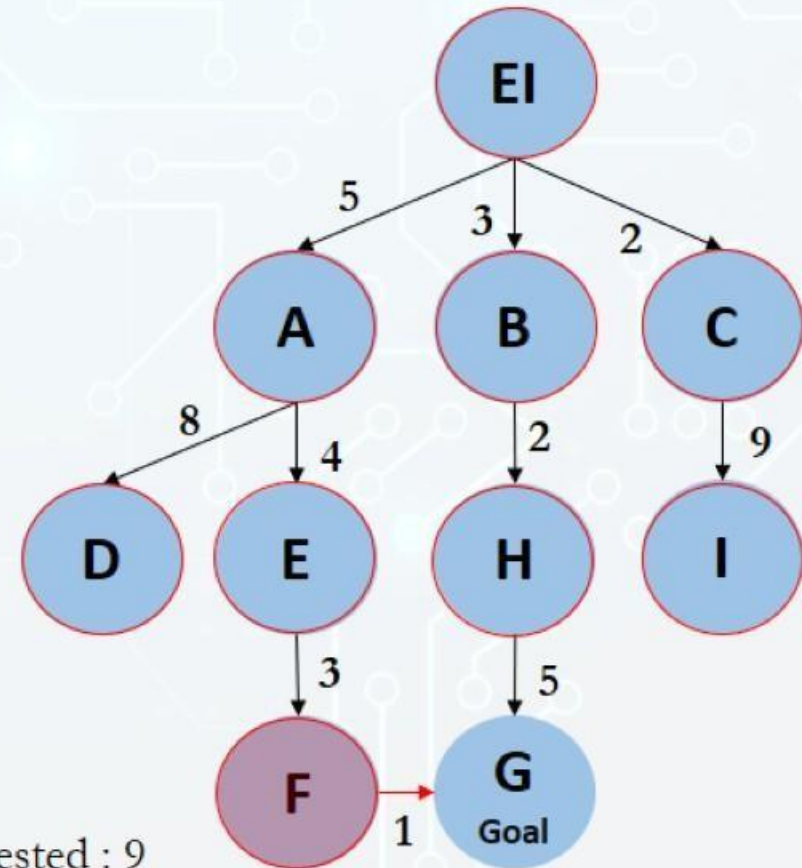


- NB nodes tested : 8
- NB expanded : 6

Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{A,B,C}
A not Goal	{B,C,D,E}
B not Goal	{C,D,E,H}
C not Goal	{D,E,H,I}
D not Goal	{E,H,I}
E not Goal	{H,I,F}
H not Goal	{I,F,G}
I not Goal	{F,G}
F not Goal	{G}

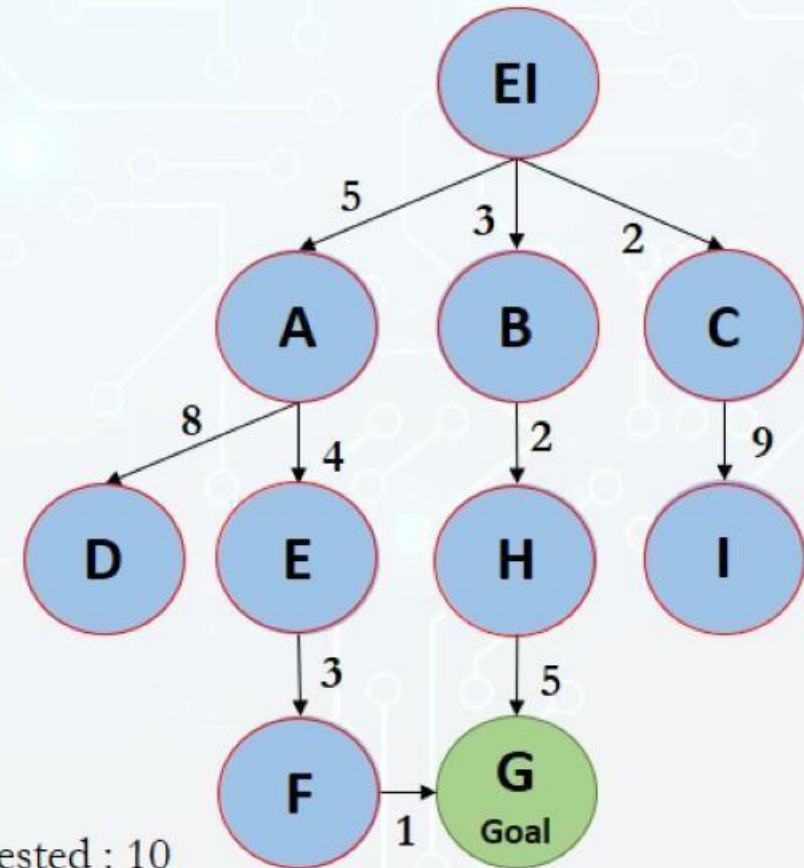


- NB nodes tested : 9
- NB expanded : 7

Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

expand.node	Nodes list
	{E1}
E1 not Goal	{A,B,C}
A not Goal	{B,C,D,E}
B not Goal	{C,D,E,H}
C not Goal	{D,E,H,I}
D not Goal	{E,H,I}
E not Goal	{H,I,F}
H not Goal	{I,F,G}
I not Goal	{F,G}
F not Goal	{G}
G the Goal	{}

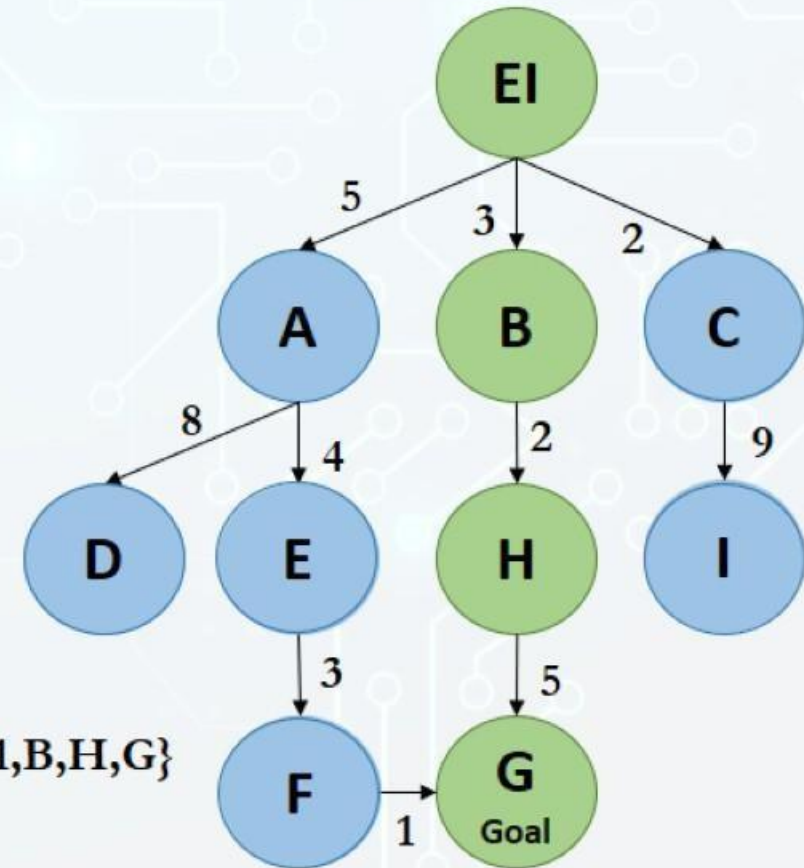


- NB nodes tested : 10
- NB expanded : 7

Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

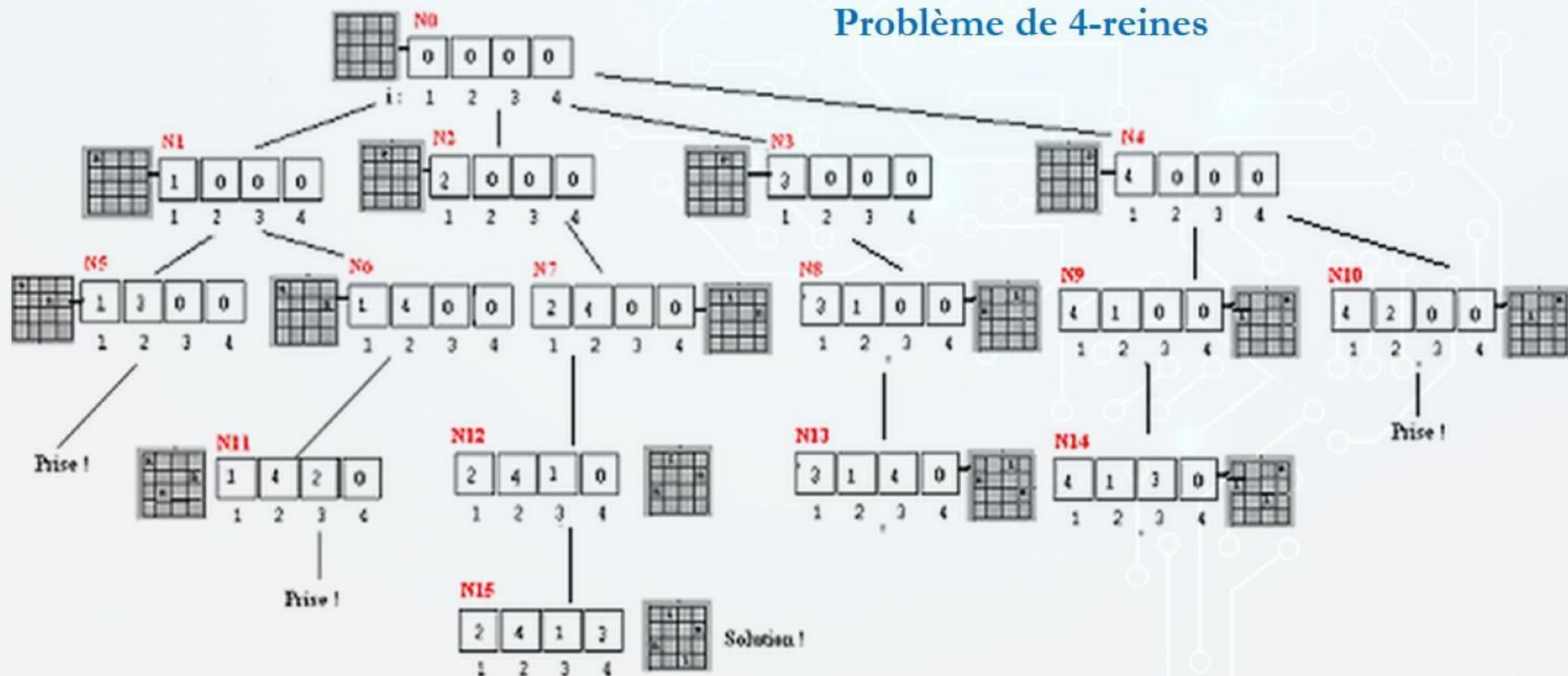
expand.node	Nodes list
	{E1}
E1 not Goal	{A,B,C}
A not Goal	{B,C,D,E}
B not Goal	{C,D,E,H}
C not Goal	{D,E,H,I}
D not Goal	{E,H,I}
E not Goal	{H,I,F}
H not Goal	{I,F,G}
I not Goal	{F,G}
F not Goal	{G}
G the Goal	{}



- Chemin : {E1,B,H,G}
- Coût : 10

Recherche non-informée

Problème de 4-reines



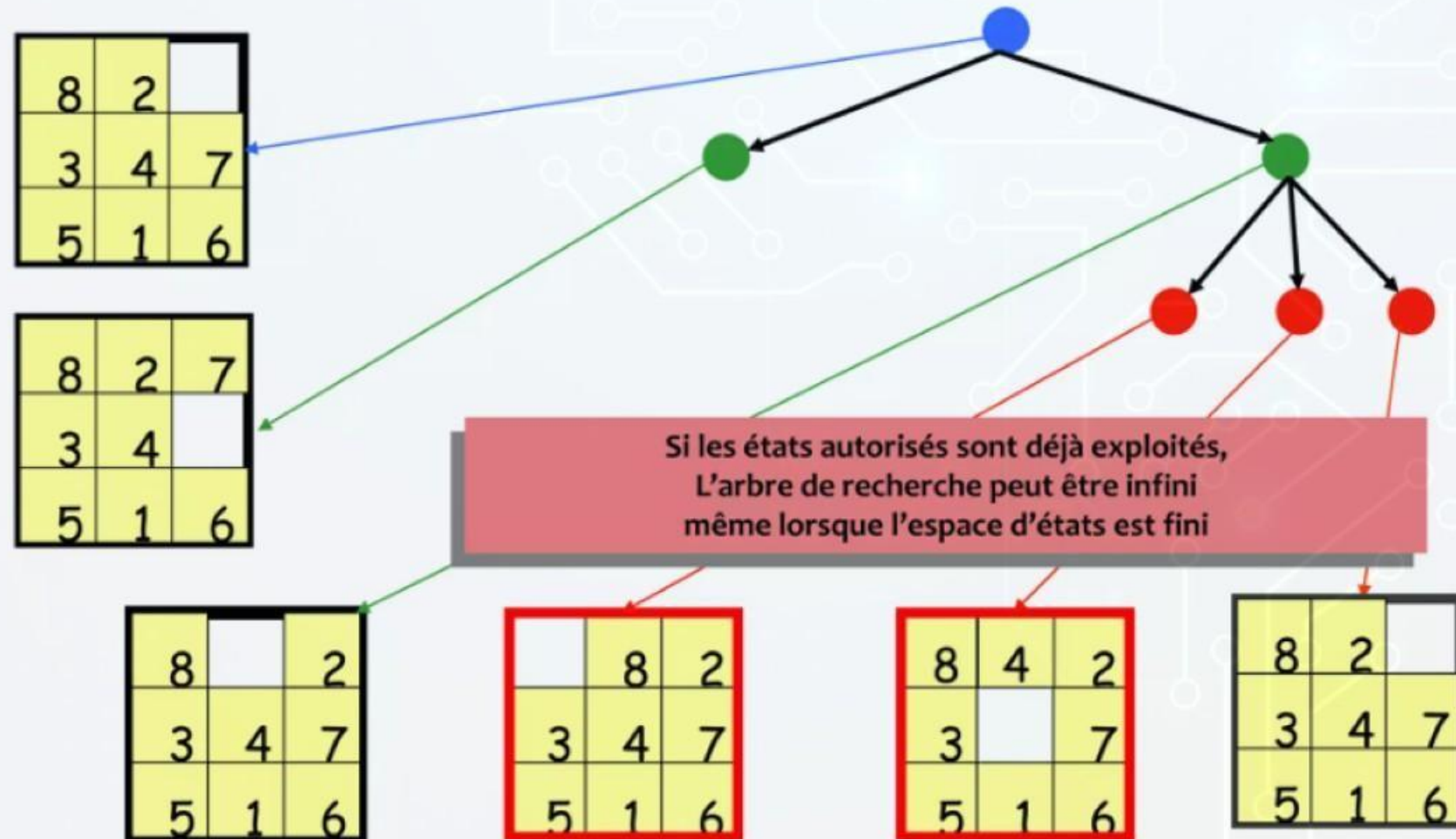
Recherche non-informée

❑ Recherche en Largeur d'abord / BFS – Breadth-First Search

▪ Propriétés

- **Complétude** : Toujours complète si le facteur de branchement b est fini. Sensibilité à l'ordre des successeurs.
- **Complexité temps** : Exponentielle, de l'ordre de $O(b^d)$. Le nombre de nœuds explorés croît rapidement avec la profondeur.
- **Complexité en espace** : $O(b^d)$. BFS doit stocker tous les nœuds d'un niveau dans une file avant de passer au niveau suivant, ce qui consomme beaucoup de mémoire.
- **Optimalité** : Oui (Si la génération de successeurs est sans coût ou à coût uniforme).

Recherche non-informée



Recherche non-informée

Un fermier doit traverser une rivière avec un loup (**W**), une chèvre (**S**) et un chou (**C**).

- **W** : Loup (Wolf)
- **S** : Chèvre (Sheep)
- **C** : Chou (Cabbage)
- **B** : Bateau (Boat)

Il a un bateau (**B**) qui ne peut transporter que lui-même et **un seul** des trois éléments à la fois.

Règles :

- Il ne peut pas laisser le loup seul avec la chèvre (le loup mangerait la chèvre).
- Il ne peut pas laisser la chèvre seule avec le chou (la chèvre mangerait le chou).

Exemple : River Test Pro



Recherche non-informée

Exemple : River Test Pro

Déterminer :

- Les états :
- L'état initial :
- Les actions :
- L'état But :

Recherche non-informée

Exemple : River Test Pro

Proposition :

- Les états : ne considérer que la coté gauche de la rivière (W,S,C,B)
- L'état initial : (W,S,C,B)
- Les actions : Traverser (B,...)
- L'état But : (-,-,-,-)

Recherche non-informée

Exemple : River Test Pro

